Laboratorio Calificado 08

2026-1

ÍNDICE

1 Instrucciones 3
2 Contexto 3
3 Objetivos 4
4 Requisitos 4
5 Entregables 5
6 Rúbrica de Evaluación 5

Laboratorio Calificado 08

Programación de un mantenimiento para la entidad 'Unidad Robótica' del Sistema de Gestión de Ensamblaje en una Línea de Producción Automatizada de Mecatrónica

1 Instrucciones

Se ha creado la tarea “Laboratorio Calificado 8” en la sección Laboratorio Calificado 8 de la página del curso-horario en PAIDEIA. Cada alumno deberá entregar un archivo comprimido (formato ZIP) con todos los archivos del desarrollo realizado en el laboratorio. El nombre del archivo comprimido deberá tener el formato:
L8_<Código del alumno de 8 dígitos>.<Extensión del archivo>
Como ejemplo, el nombre de archivo del alumno 20236969 sería “L8_20236969.zip”. Es COMPLETA RESPONSABILIDAD DE CADA ALUMNO el colocar nombres correctos a sus archivos y evitar así confusiones al momento de la calificación.

2 Contexto

Durante los laboratorios anteriores se ha desarrollado un Sistema de Gestión de Ensamblaje en una Línea de Producción Automatizada de Mecatrónica, el cual incluye:

- **Modelos** como la entidad **UnidadRobótica**, con su respectivo mantenimiento CRUD.
- **Controladores (Controllers)** que gestionan entidades con repositorios en memoria y persistencia en archivo.
- **Interfaces gráficas (GUI)** implementadas en C++/CLI con Windows Forms, que permiten realizar las operaciones CRUD.
- **Persistencia** para almacenar la información de manera permanente entre sesiones de ejecución.

En una nueva iteración del análisis del sistema, se ha identificado la necesidad de gestionar una nueva entidad denominada **EstacionTrabajo**, la cual representa los módulos físicos o celdas que forman parte de la línea de producción automatizada (por ejemplo, Estación de Soldadura, Estación de Inspección de Visión Artificial, o Estación de Empaque).

Cada estación de trabajo debe registrarse con los siguientes atributos:

- **Identificador único (id)** — Código de la estación.
- **Nombre (nombre)** — e.g., "Módulo de Soldadura Láser", "Celda de Inspección Óptica".
- **Tipo (tipo)** — e.g., "Ensamble", "Inspección", "Paletizado".
- **Estado (estado)** — e.g., "Activa", "En Configuración", "Detenida por Alerta".
- **Capacidad Máxima (capacidad)** — Número máximo de activos o tareas que puede procesar en paralelo.

El objetivo es desarrollar un mantenimiento completo (CRUD) para la entidad **EstacionTrabajo**, que permita agregar, modificar, eliminar y visualizar todas las estaciones registradas, integrando de manera limpia todas las capas del sistema (modelo, control, persistencia y GUI) ya estudiadas.

3 Objetivos

Implementar las capas necesarias del sistema, garantizando que:

- **Al iniciar el programa**, se carguen automáticamente las estaciones de trabajo almacenadas en el archivo de persistencia hacia el repositorio en memoria.
- **Las operaciones CRUD** (agregar, modificar, eliminar y consultar) de las estaciones de trabajo desde la GUI interactúen correctamente con la capa de control y de persistencia.
- **Al cerrar la aplicación**, los datos de las estaciones de trabajo se guarden de forma permanente en el archivo externo, asegurando la continuidad de la información de la planta.

4 Requisitos

a) Implementación de la capa del modelo (2 puntos)

- Definir la clase EstacionTrabajo con los atributos técnicos mencionados (ID, nombre, tipo, estado y capacidad máxima).
- Asegurar el correcto encapsulamiento de los datos e integrarla limpiamente dentro del espacio de nombres destinado a los modelos del sistema.

b) Implementación de la capa de persistencia (4 puntos)

- Definir o adaptar la clase encargada de la lectura y escritura de los datos de las estaciones en un archivo externo (e.g., formato de texto plano .txt o .csv).
- Se debe respetar estrictamente el principio de **separación de responsabilidades**, evitando acoplar la lógica de manipulación de archivos con las reglas de negocio o los elementos de la interfaz visual.

c) Implementación de la capa de control (4 puntos)

- Diseñar o adaptar el controlador (e.g., EstacionController) responsable de orquestar las operaciones del repositorio en memoria:
  - Agregar una nueva estación de trabajo.
  - Consultar el listado completo de estaciones.
- El controlador debe sincronizarse correctamente con la capa de persistencia para garantizar que los cambios impacten en el archivo de almacenamiento.

d) Implementación de la capa de presentación (GUI) (8 puntos)

- Diseñar una interfaz gráfica clara y funcional en **C++/CLI con Windows Forms** que permita al operador de planta:
  - **Agregar una estación de trabajo** completando los campos mediante elementos de entrada estándar (*TextBox*).
  - **Visualizar todas las estaciones** registradas cargando los registros de forma dinámica en un control **DataGridView**.
  - **Validar datos básicos** antes del registro (e.g., verificar que no existan campos vacíos, asegurar que el ID de la estación sea único y restringir que la capacidad máxima sea un valor numérico coherente).
  - **Mostrar mensajes de retroalimentación** claros mediante cuadros de diálogo (*MessageBox*) para confirmar el éxito de la operación o alertar sobre un error de validación (e.g., "Estación registrada exitosamente" o "Error: El ID de la estación ya se encuentra registrado").

e) Ejecución y pruebas (2 puntos)

- Validar el comportamiento integral del sistema ejecutando la siguiente secuencia de pruebas:
  - **Inserción de 2 estaciones de trabajo** con perfiles diferentes (e.g., una "Estación de Soldadura Láser" y una "Celda de Inspección Óptica").
  - **Consulta general** de todas las estaciones registradas en la línea.
  - **Verificación de la persistencia:** Reiniciar la aplicación y constatar que las estaciones ingresadas previamente se cargan de forma automática en el DataGridView.
- El informe de entrega debe incluir las capturas de pantalla detalladas que sirvan como evidencia de cada operación realizada en la GUI.

5 Entregables

El estudiante deberá presentar los siguientes archivos y evidencias a través de la plataforma virtual:

- **Código fuente completo en C++/CLI:** Todo el proyecto debe estar debidamente estructurado en sus respectivas capas (Modelo, Control, Persistencia y Presentación), limpio de archivos temporales del compilador, y con el código bien comentado para facilitar su revisión.
- **Informe de pruebas (Capturas de pantalla):** Un documento (o sección dentro del código) que incluya las capturas de pantalla de la **GUI** funcionando, evidenciando explícitamente:
  1. La inserción exitosa de las **2 estaciones de trabajo** con datos diferentes.
  2. La consulta general de la línea de producción mostrando todas las estaciones registradas dentro del *DataGridView*.

6 Rúbrica de Evaluación

Puntaje Máximo: 20 puntos

Criterio	Excelente (100%)	Bueno (80%)	Regular (60%)	Deficiente (40%)	No Cumple (0%)
a) Capa del Modelo (2 pts)	La clase Máquina está completamente definida e integrada al modelo.	Clase definida, pero con pequeños errores en atributos o métodos.	Clase incompleta o con atributos erróneos.	Clase mal estructurada o incoherente.	No se implementó la clase.
b) Capa de Persistencia (4 pts)	Persistencia totalmente funcional (lectura y escritura).	Funciona con errores menores o sin modularidad total.	Solo guarda o solo carga.	Guarda o carga incorrectamente.	No se implementó persistencia.
c) Capa de Control (4 pts)	CRUD 'Agregar' y 'Consultar todas' completamente funcionales e integrados con modelo y persistencia.	Controlador funcional con pequeños errores.	Solo una operación CRUD funciona correctamente.	Controlador con errores graves o incoherente.	No se implementó controlador.
d) Capa de Presentación (8 pts)	GUI funcional, intuitiva, con validaciones y visualización en grid.	GUI funcional, con pequeños errores de diseño o validación.	GUI incompleta (falta grid o formulario).	GUI con errores graves o poco funcional.	No se implementó GUI.

e) Ejecución y Pruebas (2 pts)	Operaciones ejecutadas correctamente, datos persistentes, capturas claras.	Funcionalidad completa con un error menor o falta de una captura.	Solo una operación funcional.	Ejecución parcial o sin persistencia.	No se realizaron pruebas.